Phishing Digital Twin - Project Explanation

A plain-English guide to what I built, how it works, and what each file does. Every file name and number here was taken from the code.

1. What is my project?

What it does. A fictional hospital has 25 members of staff. The program sends them emails - some are fake (phishing), some are real. Each person decides: is this spam or not? A **digital twin** (a software copy of a person) watches those answers and learns to predict what that person will do next. It is then tested on emails that did not exist when it was learning.

Nothing is real. Fake hospital, fake names, and every email address ends in `.example`, a reserved ending that cannot exist on the internet. No email is ever sent anywhere.

The problem it solves. Security training is usually the same for everybody. My project asks a better question: *can we predict which specific person will fall for which specific trick, without being told anything secret about them?* If you can predict it, you can train the right people on the right weakness.

Technologies I used, and why

Technology	What it is	Why I used it
Python	A programming language	Everything is written in it. Easy to read.
Streamlit	Turns Python into a website	I write Python, it makes the web page. No HTML, CSS or JavaScript needed.
SQLite	A database inside one file	Lets me ask questions like 'which emails fooled nurses?' in one line. Comes free with Python.
pandas	Works with tables of data	Builds the tables shown on screen.
Altair	Draws charts	Makes the bar charts on the Dashboard.
scikit-learn	Machine learning	Trains a model to compare against my simple counting model.

The architecture in one sentence

It is **one Python program** split into three folders by job: `frontend/` decides how things look, `backend/` decides what happens, and `database/` stores it.

```
app.py the web page you see (8 tabs)
frontend/ colours, styling, mobile layout (2 files)
backend/ all the thinking (18 files)
database/ SQLite storage (2 files)
data/ the saved files - emails, people, every answer
```

Why split it up? So each part can be changed without breaking the others. I can change a colour without touching how the model works.

2. Frontend - what you see

The frontend is the part on screen: buttons, tables, charts. In most websites this is written in HTML and JavaScript. Streamlit lets me write it in Python instead.

File	Lines	What it does
app.py	1,416	The whole web page. Draws all 8 tabs, and handles every button.
frontend/theme.py	53	The colours and chart styling, kept in one place.
frontend/responsive.py	117	CSS so the page works on a narrow window or a phone.

What happens when someone opens it

```
1. I type: python3 -m streamlit run app.py
2. Streamlit starts a small web server on my computer (localhost:8501)
3. The browser opens it
4. app.py loads 25 people and 51 emails from disk
5. It replays 11,544 past answers to rebuild what the twin knows
6. It draws the 8 tabs
```

The 8 tabs

Tab	What it does
Dashboard	The whole workforce at a glance - who clicks, and which trick works best.
Live	Emails arrive one at a time, on screen, with the numbers updating.
Inbox	YOU play a staff member. Judge an email, get an explanation.
Person	One person's report card, split by spam accuracy and genuine accuracy.
All history	Every answer ever recorded. Filterable, downloadable as CSV.
Run	The same loop as Live but silent and fast - up to 1,000,000 deliveries.
Model	Trains a machine-learning model and races it against my simple model.
Emails	Builds a brand-new email and shows exactly what was changed.

How a button works

In Streamlit, a button is one line of Python. When you click it, Streamlit **runs the whole file again from the top**, and this time the button returns True:

```
said_spam = st.button("Spam") # app.py line 745
if said_spam: # line 748
twin.update(...) # line 756 - the backend is called here
```

That re-run is the important idea. There is no separate page load like a normal website. The script runs again, top to bottom, every time you touch anything.

3. Backend - the thinking

The backend is the part you cannot see: the rules, the maths, the saving. It lives in `backend/` - 18 files. None of them import Streamlit, so they also work from a plain script or a test.

File	What it does
twin.py	THE MODEL. Counts what each person does and predicts what they will do next.
engine.py	One delivery: pick an email, predict, get the answer, update. Used by Live AND Run.
reader.py	Decides spam or not spam by INSPECTING the email - sender, link, wording.
persona.py	Loads the 25 staff members from a file.
generator.py	Chooses which email to send next.
email_builder.py	Builds brand-new emails out of parts.
modifications.py	17 ways to make an email look fake (fake sender, hidden link, urgency...).
entities.py	Finds changeable values in an email (dates, ticket numbers) and swaps them.
feedback.py	Writes the explanation shown after an answer.
ml_model.py	The machine-learning pipeline, using scikit-learn.
store.py	Saves and loads the files on disk.

How the frontend talks to the backend

There is no API. An API is usually needed when the web page and the server are separate programs that must send messages over the internet. Mine are the **same program**, so app.py just calls the functions directly, like any Python code:

```
from backend.twin import TwinPredictor # app.py line 27
twin.update(person_id, email, action) # app.py line 756

No HTTP. No JSON. No REST. Just a function call.
```

Say this clearly if asked - it is a real design decision, not something missing. One program is simpler, faster, and right for a research tool used by one person.

The model, in one line

This is the whole prediction model, in `backend/twin.py` line 312:

```
def p_click(a, b):
if a + b == 0: return 0.0
return a / (a + b) # P(click) = a / (a + b)
```

a = how many times this person clicked a fake email. **b** = how many times they reported one. Both start at 0. If someone clicked 3 and reported 1, the model says 3/(3+1) = 0.75, so a 75% chance they fall for the next one of that type. That is the whole model - just counting.

The database

SQLite - a database that lives in one file, `data/simulation.db` (2.5 MB). Six tables:

Table	Rows	What it holds
deliveries	11,544	Every answer ever given
emails	51	The fixed emails
people	25	The staff
hidden_probs	150	The secret answer key (the model is forbidden from reading it)
red_flags	125	The clues that give each fake email away

The database is a **copy**, not the original. The real record is `data/history_log.jsonl` - a file that only ever gets lines ADDED, never changed. If the database is deleted, I rebuild it from that log.

4. Complete flow - one email, start to finish

Here is exactly what happens when you press **Spam** in the Inbox tab:

Step	Where	What happens
1	You	You click the Spam button.
2	app.py:745	Streamlit re-runs the whole file. said_spam is now True.
3	app.py:752	'Spam' is turned into the action "report".
4	backend/twin.py:330	twin.update() runs. Because it was reported, b goes up by 1.
5	backend/twin.py:312	The new probability is worked out: a / (a + b).
6	backend/twin.py:137	The answer is written to the log file and forced to disk straight away.
7	backend/feedback.py	The explanation is built - what gave the email away, plus your history.
8	app.py	Streamlit redraws the page with the feedback and the new number.

Why step 6 matters: the answer is saved before anything else can go wrong. If the program crashed one line later, the answer would still be there.

5. How to demonstrate it (3-5 minutes)

Open these files in VS Code before you start: app.py, backend/twin.py, backend/engine.py, backend/reader.py.

Say: "I built a system that predicts which employee will fall for which kind of fake email. It is a digital twin - a software model of each person that learns only by watching what they do."

1. Show the Dashboard. "These are 25 staff and 11,544 recorded answers. You can see low-awareness staff click far more than high-awareness staff."

2. Show the Live tab, press Start. "Each line is one email arriving at one person. The probability on the right updates after every answer."

3. Open backend/twin.py, line 312. "This is the entire model. a divided by a plus b. a is clicks, b is reports. No machine learning - just counting."

4. Show the Model tab. "Here I trained a random forest and raced it against that one line. The random forest wins by less than one percent, which tells me the simple model is doing almost all the work."

5. Show the Emails tab. "Emails are built fresh each time, so the model can be tested on emails that did not exist when it learned."

6. Ten questions you may be asked

Question	Short answer
Do you have a REST API?	No. The web page and the logic are one program, so app.py calls backend functions directly. An API is only needed when they are separate programs.
What database do you use?	SQLite, in data/simulation.db. It is a copy of the real record, which is an append-only log file.
Why Streamlit, not Django or Flask?	Those need HTML and JavaScript. Streamlit lets me write the whole interface in Python, which suits a research tool.
How does the model learn?	It counts. a = clicks, b = reports. Prediction is a/(a+b). Both start at 0.
Is that machine learning?	The counting model is not. I also built a real ML pipeline in backend/ml_model.py with scikit-learn, to compare.
Which is better?	The random forest beat the counting model by about 0.6 percent. Almost all the work is done by the simple formula.
What are hidden_probs?	Each person's secret click chance. The model is NEVER allowed to read them - they exist only to grade it afterwards, like an answer key.
Could you send real phishing emails?	No. Every address ends in .example, a reserved ending that cannot exist. There is no sending code at all.
What happens if the program crashes?	Every answer is written to disk immediately. At worst the last line is lost, and the code skips damaged lines when reloading.
What is the biggest limitation?	No real humans have used it yet. The answers come from a rule-based reader, so the accuracy numbers show the system works - not how real people behave.